MYSQL ASSIGNMENT 2

Querying Data

SQL Queries, Filtering, Sorting, Aggregate Functions, Grouping and Joins

Student Name: JASMINE S

Course / Class: DATA Analytics

Tool Used: MySQL Workbench

1. Introduction

This assignment practices SQL queries for retrieving and modifying employee data. It uses filtering, aliases, sorting, limiting results, aggregate functions, grouping, HAVING conditions, and joins across the Employees, Departments, and Location tables.

2. Objectives

• Retrieve distinct values and display columns with aliases.

• Filter employee records using WHERE and comparison conditions.

• Update a missing designation safely.

• Sort and limit query results.

• Use SUM, MIN, MAX, and AVG aggregate functions.

• Group records and filter groups using HAVING.

• Combine related records using INNER JOIN, LEFT JOIN, and RIGHT JOIN.

3. SQL Queries and Explanations

Question 1: Distinct Salaries

Retrieve each salary value only once.

```
SELECT DISTINCT salary
FROM employees;
```

Question 2: Column Aliases

Display age and salary with the requested headings.

```
SELECT
    age AS Employee_Age,
    salary AS Employee_Salary
FROM employees;
```

Question 3: WHERE Clause and Operators

Find employees earning more than ■50,000 who were hired before 1 January 2016.

```
SELECT *
FROM employees
WHERE salary > 50000
  AND hire_date < '2016-01-01';
```

Question 4: Find and Update Missing Designation

First identify records with a missing designation. Verify the ID returned by the SELECT before updating; 5004 is an example based on the supplied dataset.

```
SELECT employee_id, employee_name, designation
FROM employees
WHERE designation IS NULL;

UPDATE employees
SET designation = 'Data Scientist'
WHERE employee_id = 5004
  AND designation IS NULL;
```

Question 5: ORDER BY

Sort by department ID ascending, then salary descending.

```
SELECT *
FROM employees
ORDER BY department_id ASC, salary DESC;
```

Question 6: LIMIT

Display the first five employees hired during 2018, ordered by hire date.

```
SELECT *
FROM employees
WHERE hire_date >= '2018-01-01'
  AND hire_date < '2019-01-01'
ORDER BY hire_date ASC
LIMIT 5;
```

Question 7: SUM of Finance Salaries

Calculate the total salary for employees assigned to Finance.

```
SELECT SUM(e.salary) AS Total_Finance_Salary
FROM employees e
JOIN departments d
    ON e.department_id = d.department_id
WHERE d.department_name = 'Finance';
```

Question 8: Minimum Age

Find the minimum age in the Employees table.

```
SELECT MIN(age) AS Minimum_Age
```

```
FROM employees;
```

Question 9: Maximum Salary by Location

Find the highest salary for each location. This database uses location_name for the location text.

```
SELECT
    l.location_name,
    MAX(e.salary) AS Maximum_Salary
FROM location l
JOIN employees e
    ON l.location_id = e.location_id
GROUP BY l.location_id, l.location_name;
```

Question 10: Average Salary for Analyst Designations

Calculate the average salary for each designation containing the word “Analyst”.

```
SELECT
    designation,
    AVG(salary) AS Average_Salary
FROM employees
WHERE designation LIKE '%Analyst%'
GROUP BY designation;
```

Question 11: Departments With Fewer Than Three Employees

Include departments with no employees and return those with fewer than three employees.

```
SELECT
    d.department_id,
    d.department_name,
    COUNT(e.employee_id) AS Total_Employees
FROM departments d
LEFT JOIN employees e
    ON d.department_id = e.department_id
GROUP BY d.department_id, d.department_name
HAVING COUNT(e.employee_id) < 3;
```

Question 12: Female Employees With Average Age Below 30 by Location

Return locations where the average age of female employees is below 30. This database uses location_name for the location text.

```
SELECT
    l.location_name,
    AVG(e.age) AS Average_Age
FROM location l
JOIN employees e
    ON l.location_id = e.location_id
WHERE e.gender = 'F'
GROUP BY l.location_id, l.location_name
HAVING AVG(e.age) < 30;
```

Question 13: INNER JOIN

List employee names, designations, and department names for employees linked to a department.

```
SELECT
    e.employee_name,
    e.designation,
    d.department_name
FROM employees e
INNER JOIN departments d
    ON e.department_id = d.department_id;
```

Question 14: LEFT JOIN

List every department and its employee count, including departments with no employees.

```
SELECT
    d.department_name,
    COUNT(e.employee_id) AS Total_Employees
FROM departments d
LEFT JOIN employees e
    ON d.department_id = e.department_id
GROUP BY d.department_id, d.department_name;
```

Question 15: RIGHT JOIN

Display all locations with employee names. Locations without a matching employee show NULL for employee_name.

```
SELECT
    l.location_name,
    e.employee_name
FROM employees e
RIGHT JOIN location l
    ON e.location_id = l.location_id
ORDER BY l.location_name, e.employee_name;
```

4. Conclusion

This assignment demonstrates how SQL can retrieve, filter, update, sort, summarize, group, and relate employee data. Aggregate functions provide summary information, while JOIN operations combine data from related tables. Query results should be checked in MySQL Workbench after execution.

5. Submission Note

If required by your instructor, add MySQL Workbench screenshots showing successful execution and query output. The SQL examples assume the table names employees, departments, and location, and the location-name column is location_name. In Question 4, verify the employee ID returned by the initial SELECT before running the UPDATE.